BLENTINKERKIT

Enrico Sailis

enricosailis@gmail.com



COS'E' BLENTINKERKIT

Blentinkerkit è un insieme di applicazioni scritte in Python e in C++ che consentono di pilotare il braccio robotico Arduino Tinkerkit attraverso il software di animazione gratuito Blender. L'obiettivo è quello di far prendere al braccio una spugnetta per disporla poi in un punto voluto in modo automatico. Tutto ciò è reso possibile grazie all'add-on blendixserial di Blender che consente di interfacciare Blender mediante una porta USB del PC in uso, con la scheda Arduino1 che pilota il braccio. Attraverso una videocamera collegata al PC che inquadra l'area in cui è collocata la spugnetta, un software autonomo in python fotografa la scena e dall'analisi della fotografia un software python in Blender crea i comandi di animazione del braccio per prendere la spugna e disporla in un punto voluto. Nel breve Video_lab.mp4 si vede come agisce il braccio robotico sotto il controllo di Blender.

Il **codice** utilizzato, i video e i file di supporto sono nel repository blentinkerkit su GitHub.

FONDAMENTALI PER L'ANIMAZIONE DEL BRACCIO

Scarica il software gratuito di animazione Blender, installalo nel tuo PC Windows e avvialo. **Il tuo PC deve avere almeno 8 GB di ram e un processore con almeno 4 core** per poter far girare Blender. Per i primi rudimenti sul sull'utilizzo di Blender ti puoi avvalere di Gemini, il sistema IA di Google Chrome.

Crea una cartella con il nome: blentinkerkit e la sottocartella python, in questa ultima carica i seguenti file che si trovano nella sottodirectory python del repository blentinkerkit:

AnimazioneBraccio_win.py (contiene il codice python per l'animazione del braccio)

ElaborazioneImmagineBraccio_win.py (contiene il codice python per elaborare immagini)

ImageFromWebCam.py (contiene il codice python per acquisire immagini)

Nella tua directory principale blentinkerkit carica i seguenti file zippati:

L'add-on blendixserial.zip

Il file Blender: Blentinkerkit_win_blend.zip

e infine carica nella cartella blentinkerkit il file di supporto: Quadretti_colorati.png

Dopo aver spostato il file blentinkerkit_win.blend.zip, decomprimilo e copia il file blentinkerkit_win.blend nella directory principale blentinkerkit. Esso è il file principale contenente le animazioni. L'estensione blend denota che si tratta di un file per il programma di animazione Blender. Avvialo da Blender ed esegui l'animazione in esso contenuta con il tasto Play Animation. Mettendo la vista dall'alto (asse z) puoi spostare la spugnetta selezionandola e poi ricollocandola con il tasto G (Grab) mantenendola tra i due cerchi che individuano la regione accessibile al braccio, poi fai rigenerare l'animazione per la presa con il comando Run Script nel Workspace Scripting, dopo torna nel Workspace Layout e riavvia l'animazione per vedere i nuovi movimenti del braccio. Se vuoi puoi spostare il cerchietto Target dove avverrà il deposito della spugnetta con la stessa modalità con cui si sposta la spugnetta.

UTILIZZO DELLA VIDEOCAMERA

Passiamo ora all'utilizzo di una videocamera che inquadra la scena dove è presente la spugnetta per fare una foto e determinare da questa la posizione della spugnetta, note le dimensioni in pixel della foto e le dimensioni reali dell'area ripresa.

E’ bene usare un piano di lavoro dove agisce il braccio robotico di colore bianco e con le dimensioni grosso modo uguali a quelle del piano quadrettato in Blender di dimensioni 80 cm x 50 cm.
Per la determinazione della posizione della spugnetta rispetto al braccio servono diversi parametri tra cui le dimensioni reali della scena ripresa dalla videocamera che ovviamente dipendono dalla collocazione della fotocamera. Si tenga presente che l’origine del sistema di riferimento della posizione della spugnetta deve essere il centro del braccio (Base) con gli assi x e y orientati come nella Fig. 4. parallelamente ai lati dei quadretti della base d’appoggio del braccio. Il lati di questi quadretti nella scala di Blender valgono 10 cm e devono corrispondere a 10 cm reali nella foto.
I parametri da determinare per individuare la spugnetta sono: percorso del file della foto snapshot.jpg; larghezza e altezza reali dell’immagine ripresa; dimensioni in pixel della foto e il range dei colori RGB della spugnetta (mattoncino) da individuare presente nella foto; origine del sistema di riferimento.
Si tenga presente che il sistema di riferimento usato dal software per le analisi della immagine ha per assi x, y rispettivamente il bordo superiore e il bordo sinistro della immagine con l’asse y rivolto in basso e l’asse x rivolto verso destra. E’ possibile sperimentare i parametri che dovrà assumere la funzione rilevaMattoncino( ) che si trova nel file AnimazioneBraccio_win.py con il programma python

ElaborazioneImmagineBraccio_win.py

che fornisce le coordinate del centro della spugnetta (mattoncino) sia in coordinate sia in pixel. Fare con questa applicazione diversi tentativi di determinazione del centro della spugnetta rispetto al braccio (come nella animazione in Blender) prima di inserire i parametri più idonei nel programma rilevaMattoncino( ) di fondamentale importanza per l’animazione del braccio.
Una volta che il programma ElaborazioneImmagineBraccio_win.py fornisce misure corrette impostare i parametri individuati all’interno del funzione rilevaMattoncino( ) che si trova all’interno del file AnimazioneBraccio_win.py accessibile in Blender dal Workspace Scripting. Questa funzione parte dalla riga 1234 del programma. Nello stesso file di animazione togliere il simbolo # dalla riga 1303 per abilitare la chiamata a questa funzione.
Ogni volta che si sposta la spugnetta bisogna riposizionarla in un punto raggiungibile dal braccio tra i due cerchi nel layout di Blender con centro nel centro della base del braccio, scattare una foto con il programma ImageFromWebCam.py e salvarla. Dopo lanciare da Blender il programma di AnimazioneBraccio_win.py precedentemente modificato dal Workspace Scripting, il quale, dopo aver analizzato la foto come fa il programma ElaborazioneImmagineBraccio_win.py, crea le animazioni. Dopo lanciare l’animazione dal Workspace Layout.
Per eseguire lo script per l’uso della videocamera e per fare la foto occorre installare l’IDLE python e poi avviarlo dal terminale di windows con il comando *python -m idlelib* (con questa modalità di avvio dell’IDLE python, i programmi python avviati dall’IDLE potranno usare la videocamera collegata alla porta USB del PC). Dopo avere avviato l’IDLE python dal terminale siamo in grado di utilizzare la videocamera collegata alla porta USB lanciando dall’IDLE il programma python

ImageFromWebCam.py

e fare con questo la fotografia che sarà salvata con il nome snapshot.jpg nella stessa sottocartella che contiene il programma. Anche il programma ElaborazioneImmagineBraccio_win.py deve essere lanciato dall'IDLE di python.

INTERFACCIAMENTO DI BLENDER COL BRACCIO TINKERKIT

INTERFACCIAMENTO BLENDIXSERIAL DELL'ANIMAZIONE BLENDER CON ARDUINO UNO

Per interfacciare le animazioni del braccio con il braccio fisico bisogna installare in Blender l'add-on blendixserial. Puoi vedere come si fa visitando il sito dell'autore. In Blender gli oggetti Cube chiamati Cubi_Dati_nome_osso, uno per ognuna delle 6 ossa governano le rotazioni del braccio, reale grazie alle animazioni realizzate tramite il codice python nel file AnimazioneBraccio_win.py collegato al file blentinkerkit_win.blend di Blender nel Workspace Scripting. I Cube_Dati ruoteranno come le ossa associate e queste rotazioni, grazie all'add-ons blendixserial verranno trasmesse tramite la porta seriale USB alla interfaccia software apposita, installata sulla scheda Arduino1, serviranno per pilotare il movimento del braccio fisico. La loro presenza serve ad ovviare ad una limitazione dell'add-on blendixserial che non consente di catturare il movimento di ossa ma solo di oggetti.

INTERFACCIAMENTO DI ARDUINO UNO CON BLENDER

Installa sul tuo computer il software Arduino IDE quindi installa in questa IDE la libreria per l'interfacciamento con Blender blendixserial-arduino-2.1.0.zip che è stata modificata perché possa lavorare con 6 oggetti anziché 3 di default (per farlo ho decompresso la libreria originale e ho impostato nei file di configurazione blendixserial_config.h e blendixserial_config.h il parametro BLENDIXSERIAL_MAX_OBJECTS 6 . La libreria modificata può essere scaricata dalla cartella Arduino del repository Blentinkerkit su GitHub .
Per utilizzare questa libreria già modificata una volta scaricata dalla cartella Arduino del repository su Github Blentinkerkit segui le seguenti istruzioni per la sua installazione su Arduino IDE:

Apri Arduino IDE e vai su
Sketch > Include Library > Add .ZIP Library…
Seleziona il file blendixserial-arduino-2.1.0.zip

Ora sempre dal repository Blentinkerkit scarica lo sketch blendixserial_braccio_control_win.ino e salvalo in una cartella con lo stesso nome e avvia l'IDE di Arduino. Carica il file appena scaricato nell'IDE Arduino e installalo sulla scheda Arduino 1 dopo averla connessa fisicamente con il tuo PC su una porta USB e averla selezionata sull'IDE. Lo sketch salvato sulla scheda Arduino 1 rimarrà anche dopo il distacco di questa dalla porta USB del PC. Ora che le interfacce tra Blender e Arduino sono installate possiamo governare il braccio con Blender. Per il momento limitiamoci a

spostare in Blender la spugnetta da prendere e a creare i frames dello spostamento del braccio avviando lo script python nel Workspace Scripting per la creazione delle animazioni per la presa, il trasporto e il deposito della spugnetta. Per far inviare a Blender i movimenti del braccio ad Arduino 1, occorre chiudere l'IDE Arduino e dal pannello di blendixserial connettere Blender con la porta seriale su cui vi è la scheda Arduino 1, con baud rate di 115200, fare clic sul pulsante Start Movement quindi impostare Mode in Send e Data Format in Protocol, quindi avviare l'animazione.

COME CREARE BLENTINKERKIT

Se vuoi creare tu il braccio in Blender e vuoi fargli pilotare il braccio fisico o fare solo delle animazioni per fargli prendere e spostare un spugnetta, segui questa guida passo passo.



Fig. 1

ASSEMBLAGGIO E ANIMAZIONE DEL BRACCIO IN BLENDER

CARICAMENTO DEI FILE STL DEI COMPONENTI DEL BRACCIO

Scarica i file in formato STL degli elementi del braccio tinkerkit. Per farlo recati nel sito grabcad.com In alternativa e più velocemente puoi scaricare il file zippato che li contiene Tinkerkit-stl.zip che si trova nella cartella Tinkerkit-stl situata nel repository Blentinkerkit su Github.

ASSEMBLAGGIO DEI COMPONENTI DEL BRACCIO IN BLENDER

Importa in Blender i file in formato STL delle componenti del braccio e assemblali. Nelle articolazioni del braccio, Base, Arm, Forearm, Wrist_vertical sono usati servomotori di tipo SR 431 mentre nelle articolazioni Wrist_rotation e Gripper invece sono usati i servomotori di tipo SR 311. Dopo l'assemblaggio assicurati che il braccio disegnato abbia le dimensioni di quello reale ovvero che la distanza sull'asse di rotazione verticale del primo snodo (Base) al primo snodo del braccio (Arm) valga 7,5 cm, le distanze tra gli snodi del braccio (Arm) e dell' avambraccio (Forearm) valgono entrambe 12,5 cm, e che la distanza tra il primo snodo del polso (Wrist_rotation) e la punta della pinza valga 18,5 cm. Per far ciò occorre scalare tutti gli elementi del braccio dello stesso valore percentuale. Salva il file di Blender con il nome Tinkerkit e mettilo in una cartella vuota dove poi caricherai gli altri file che creerai successivamente. Di seguito riporto suggerimenti riguardo le funzioni dei comandi Blender da me utilizzati nell' assemblaggio.

Se richiedi al motore di ricerca Google basato sull'IA Gemini, il funzionamento del comando Blender Grab (Afferra), oppure il funzionamento del comando Blender Scale (Scala), ti informerà sul modo di usare questi comandi in dettaglio. Comunque puoi vedere i video dell'assemblaggio che ho fatto dove puoi notare che in basso a sinistra dello schermo, mentre scorre il video passo dopo passo, c'è l'immagine del mouse con evidenziati i tasti premuti e le lettere del comando richiamato. Purtroppo non viene segnalata la rotazione della rotella del mouse.
Vedi i Video dell'assemblaggio 1-10

ANIMAZIONE DEL BRACCIO IN BLENDER

Crea le ossa Base, Arm, Forearm, Wrist_vertical, Wrist_rotation e Gripper per il controllo del braccio. Guarda i video per i dettagli della creazione dell'ossatura.
Vedi i video di animazione 1-8

Colora il braccio e aggiungi il piano di appoggio e la spugnetta.
Vedi il video di animazione 9 sulla coloritura e sull'aggiunta del piano e della spugnetta.

Inserisci il file AnimazioneBraccio_win.py in Blender nel workspace Scripting. Il file AnimazioneBraccio_win.py contiene il codice python per creare le animazioni del trasporto della spugnetta con il braccio. Prima devi rinominare le ossa con il nome degli elementi che guidano, come nella Fig. 1. Poi devi resettare (azzerare gli angoli delle ossa). Infine testa la simulazione

della presa e trasporto della spugnetta variando la posizione di questa nel piano (tra le due circonferenze segnate sul piano) e il punto dove spostarla (target) anch'esso tra le due circonferenze sul piano. Vedi i video di animazione 10-12.

LA MATEMATICA DELLE ROTAZIONI

ROTAZIONE INTORNO ALL'ASSE Z

Consideriamo un sistema di riferimento cartesiano R nello spazio, con gli assi x, y, z orientati come in figura, dove **i, j**, **k** sono i versori degli assi, x,y, z, ovvero sono vettori unitari orientati come gli assi cartesiani associati. Consideriamo un punto P nello spazio avente coordinate (x_0, y_0, z_0) nel sistema di riferimento considerato. Vogliamo ora calcolare le coordinate di questo punto (x_1, y_1, z_1) in un nuovo sistema di riferimento cartesiano R' che è ruotato rispetto al primo di un angolo θ_1 in senso antiorario rispetto all'asse z, per un osservatore disposto come l'asse z e con i

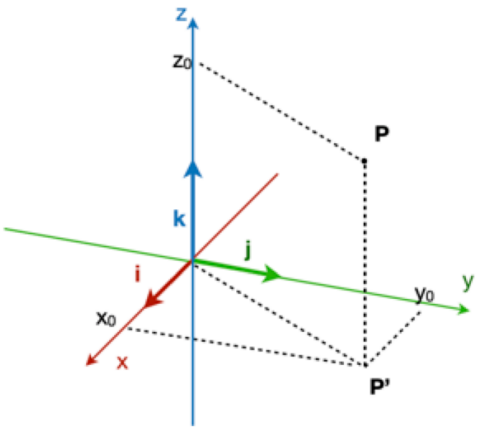


Fig. 2a

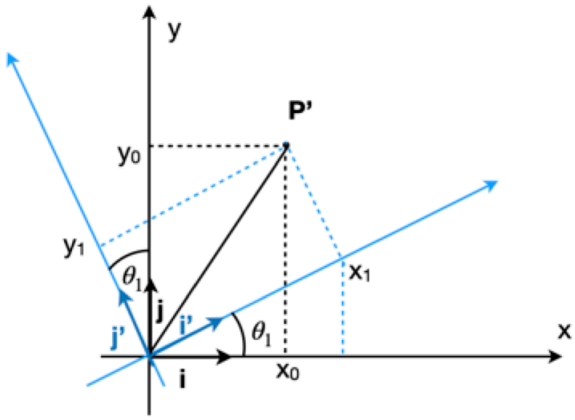


Fig. 2b

piedi sul piano x, y. Nella rotazione l'origine del sistema R' non si sposta rispetto a quella di R. La coordinata z di **P** rispetto al nuovo sistema di riferimento è uguale a quella nel vecchio sistema ovvero $z_0 = z_1$. Per determinare le coordinate di P (x_1, y_1, z_1) nel nuovo sistema di riferimento R' analizzeremo la **Fig**. 2b, ma prima facciamo un breve ripasso sui vettori.

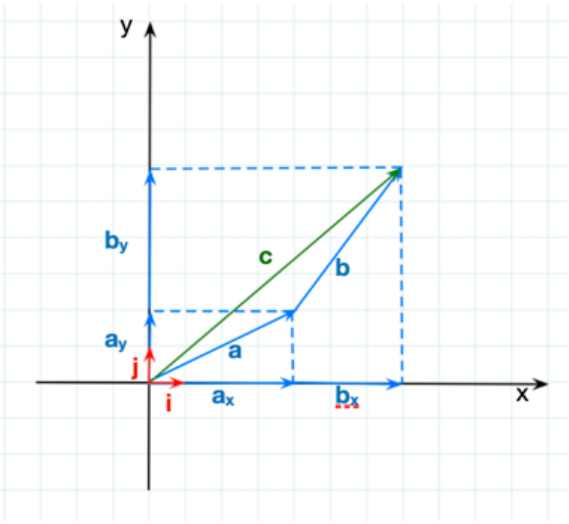


Fig. 3

Osserviamo la Fig. 3. Sono dati due vettori **a** e **b** e la loro somma **a** + **b** = **c**. La somma tra due vettori **a** e **b** si ottiene disponendo dopo il vettore **a**, il vettore **b**. La loro somma è il vettore **c** che unisce la coda di **a** con la punta di **b**. Chiamiamo $\mathbf{a_x}$ e $\mathbf{a_y}$ i vettori proiezioni ortogonali del vettore **a** sugli assi x e y rispettivamente, detti anche componenti cartesiani del vettore **a**. Facciamo altrettanto con il vettore **b**, chiamiamo $\mathbf{b_x}$ e $\mathbf{b_y}$ i vettori proiezioni ortogonali del vettore **b** sugli assi x e y rispettivamente (le componenti cartesiane del vettore **b**). Il prodotto di un vettore per un numero (prodotto scalare) si ottiene moltiplicando il modulo del vettore (la sua lunghezza) per il numero dato e cambiandone il verso se il numero è negativo, mantenendo lo stesso verso del vettore dato, se il numero per cui lo si moltiplica è positivo. Dalla Fig. 3 si evince che:

$\mathbf{a} = \mathbf{a_x} + \mathbf{a_y}$ $\quad$ $\mathbf{b} = \mathbf{b_x} + \mathbf{b_y}$ $\quad$ $\mathbf{c} = \mathbf{a} + \mathbf{b} = (\mathbf{a_x} + \mathbf{a_y}) + (\mathbf{b_x} + \mathbf{b_y}) = (\mathbf{a_x} + \mathbf{b_x}) + (\mathbf{a_y} + \mathbf{b_y})$

$\mathbf{a} = \mathbf{i}\, a_x + \mathbf{j}\, b_y$ $\quad$ $\mathbf{b} = \mathbf{i}\, b_x + \mathbf{j}\, b_y$ $\quad$ $\mathbf{c} = \mathbf{a} + \mathbf{b} = (\mathbf{i}\, a_x + \mathbf{j}\, b_y) + (\mathbf{i}\, b_x + \mathbf{j}\, b_y) = (a_x + b_x)\,\mathbf{i} + (a_y + b_y)\,\mathbf{j}$

Nelle operazioni di sopra si è tenuto conto che l'addizione tra vettori gode della proprietà associativa e commutativa come si può verificare graficamente senza troppe difficoltà. Per meglio operare in seguito, usiamo la notazione vettoriale matriciale, ossia, esprimiamo i vettori attraverso le loro componenti cartesiane disposte verticalmente, ossia:

$$a = \begin{pmatrix} a_x \\ a_y \end{pmatrix} \quad b = \begin{pmatrix} b_x \\ b_y \end{pmatrix} \quad c = \begin{pmatrix} a_x + b_x \\ a_y + b_y \end{pmatrix}$$

Passiamo ora a risolvere il problema di determinare le coordinate del punto P nel nuovo sistema di riferimento R' ruotato rispetto al sistema di riferimento R.
Indichiamo con **i**, **j**, **k** i versori degli assi x, y e z rispettivamente nel sistema di riferimento R. I versori sono vettori con modulo 1 e diretti e orientati come gli assi cartesiani a cui sono associati (vedi Fig. 3). Indichiamo invece con **i'**, **j'**, **z'**i versori degli assi x', y' e z' nel nuovo sistema di riferimento R' ruotato rispetto al primo in senso antiorario attorno all'asse z di un angolo uguale a θ_1. Dalla Fig. 2 ricordando i teoremi sui triangoli rettangoli possiamo dedurre le seguenti relazioni tra i versori dei due sistemi di riferimento:

i = **i'** cos θ_1 - **j'** sen θ_1 **j** = **i'** sen θ_1 + **j'** cos θ_1 **k** = **k'**
da cui si deduce:

x_0 **i** + y_0 **j** + z_0 **k** = x_0 (**i'** cos θ_1 - **j'** sen θ_1) + y_0 (**i'** sen θ_1 + **j'** cos θ_1) + z_0 **k** =

= x_0 cos θ_1 **i'** - x_0 sen θ_1 **j'** + y_0 sen θ_1 **i'** + y_0 cos θ_1 **j'** + z_0 **k'** =

= (x_0 cos θ_1 + y_0 sen θ_1) **i'** + (- x_0 sen θ_1 + y_0 cos θ_1) **j'** + z_0 **k'** =

= x_1 **i'** + y_1 **j'** + z_1 **k'**

dove abbiamo posto: $x_1 = x_0$ cos θ_1 + y_0 sen θ_1 , y_1 = - x_0 sen θ_1 + y_0 cos θ_1, $z_1 = z_0$

quindi in notazione matriciale :

$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} x_0 cos\theta_1 + y_0 sen\theta_1 \\ -x_0 sen\theta_1 + y_0 cos\theta_1 \end{pmatrix} = x_0 \begin{pmatrix} cos\theta_1 \\ -sen\theta_1 \end{pmatrix} + y_0 \begin{pmatrix} sen\theta_1 \\ cos\theta_1 \end{pmatrix}$$

Oppure in notazione più compatta avvalendoci della moltiplicazione tra matrici:

$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix} = \begin{pmatrix} cos\theta_1 + sen\theta_1 \\ -sen\theta_1 + cos\theta_1 \end{pmatrix} \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$$

CINEMATICA INVERSA DEL BRACCIO ROBOTICO

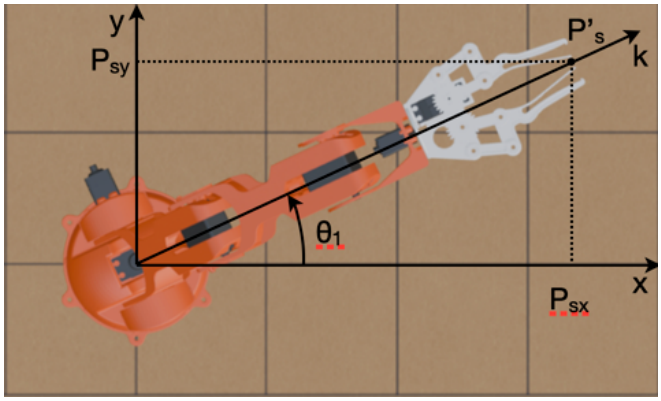


Fig. 4

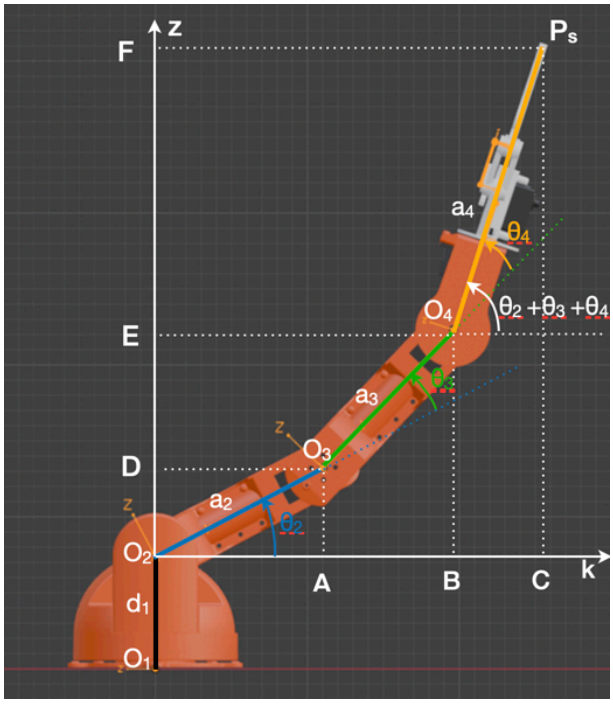


Fig. 5

La cinematica inversa del braccio robotico serve a calcolare le posizioni da far assumere ai servomotori per far raggiungere un determinato punto alla pinza. In nostro braccio tinkerkit ha 6 giunti di cui noi non useremo il quinto che serve a fare ruotare la pinza. Una volta fissata la

posizione che si vuole far raggiungere alla pinza, ovvero il centro della spugnetta, identificata dal vettore:

$$p_s = \begin{pmatrix} p_{s_x} \\ p_{s_y} \\ p_{s_z} \end{pmatrix}$$

Si procede con la soluzione della cinematica inversa calcolando gli angoli θ_1, θ_2, θ_3, θ_4. Gli angoli θ_5, θ_6, corrispondenti alla rotazione del polso e all'apertura della pinza si possono per il momento trascurare. Di seguito riportiamo gli angoli da determinare che sono rappresentati nelle figure 4 (vista dall'alto del braccio) e Fig. 5 (vista laterale del braccio). Al fine della presa della spugnetta assumeremo l'inclinazione della pinza rispetto all'orizzontale uguale a 60° verso il basso, ovvero che $\phi = \theta_2 + \theta_3 + \theta_4$ sia uguale a - 60°.

CALCOLO DELL'ANGOLO θ_1 (BASE)

L'angolo θ_1 del giunto della base si può calcolare con la formula:

$$\theta_1 = \ atan2\left(p_{s_y},\ p_{s_x}\right)$$

dove $atan2\left(p_{s_y},\ p_{s_x}\right)$ è l'arcotangente 2 ovvero l'angolo in radianti tra l'asse x e il vettore

$$\left(p_{sx},\ p_{sy}\right)$$

La funzione atan2(y, x) restituisce l'angolo in radianti come segue:

$$atan2\left(y,\ x\right) = \begin{cases} arctg(y/x) & se\ x > 0 \\ arctg(y/x) + \pi & se\ x < 0\ e\ y \geq 0 \\ arctg(y/x) - \pi & se\ x\ < 0\ e\ y < 0 \\ +\pi/2 & se\ x = 0\ e\ y > 0 \\ -\pi/2 & se\ x = 0\ e\ y < 0 \\ non\ definita & se\ x = 0\ e\ y = 0 \end{cases}$$

CALCOLO DELL'ANGOLO θ_3 (GOMITO)

Avendo imposto l'angolo $\phi = \theta_2 + \theta_3 + \theta_4$ possiamo ricavare le coordinate del polso O_4 (P_{wk}, P_{wz}) nel piano k, z (Fig. 5) con le formule:

$$p_{wz} = p_{sz} - a_4 sen\phi$$

$$p_{wk} = p_{sk} - a_4 cos\phi$$

dove dalla Fig. 4 si ricava:

$$p_{sk} = \sqrt{p_{sx}^2 + p_{sy}^2}$$

Dalla Fig. 5 si ottiene:

$$\overline{O_2A} = a_2 cos\theta_2$$

$$\overline{AB} = a_3 cos(\theta_2 + \theta_3)$$

$$\overline{O_2D} = a_2 sen\theta_2$$

$$\overline{DE} = a_3 sen(\theta_2 + \theta_3)$$

$$p_{wk} = \overline{O_2A} + \overline{AB}$$

$$p_{wz} = \overline{O_2D} + \overline{DE}$$

da cui segue:

$$p_{wk}^2 + p_{wz}^2 = (a_2 cos\theta_2 + a_3 cos(\theta_2 + \theta_3))^2 + (a_2 sen\theta_2 + a_3 sen(\theta_2 + \theta_3))^2 =$$

$$\begin{aligned} = {} & a_2^2 cos^2\theta_2 + a_3^2 cos^2(\theta_2 + \theta_3) + 2a_2 a_3 cos(\theta_2) cos(\theta_2 + \theta_3) + \\ & + a_2^2 sen^2\theta_2 + a_3^2 sen^2(\theta_2 + \theta_3) + 2a_2 a_3 sen(\theta_2) sen(\theta_2 + \theta_3) \end{aligned}$$

sapendo che

$$cos^2(\theta) + sen^2(\theta) = 1$$

si ricava:

$$p_{wk}^2 + p_{wz}^2 = a_2^2 + a_3^2 + 2a_2 a_3 cos(\theta_2) cos(\theta_2 + \theta_3) + 2a_2 a_3 sen(\theta_2) sen(\theta_2 + \theta_3)$$

$$p_{wk}^2 + p_{wz}^2 = \ a_2^2 + a_3^2 + 2a_2a_3(cos(\theta_2)cos(\theta_2 + \theta_3) + sen(\theta_2)sen(\theta_2 + \theta_3))$$

e mettendo in evidenza si ottiene:

sapendo che:

$$cos(\alpha + \beta) = cos(\alpha)cos(\beta) - sen(\alpha)sen(\beta)$$
$$sen(\alpha + \beta) = sen(\alpha)cos(\beta) + cos(\alpha)sen(\beta)$$

l'espressione

$$cos(\theta_2)cos(\theta_2 + \theta_3) + sen(\theta_2)sen(\theta_2 + \theta_3)$$

diventa:

$$cos(\theta_2)(cos(\theta_2)cos(\theta_3) - sen(\theta_2)sen(\theta_3)) + sen(\theta_2)\,(sen(\theta_2)cos(\theta_3) + cos(\theta_2)sen(\theta_3)) =$$
$$= cos^2(\theta_2)cos(\theta_3) - sen(\theta_2)sen(\theta_3)cos(\theta_2) + sen^2(\theta_2)cos(\theta_3) + sen(\theta_2)sen(\theta_3)cos(\theta_2) =$$
$$= cos^2(\theta_2)cos(\theta_3) - sen(\theta_2)sen(\theta_3)cos(\theta_2) + sen^2(\theta_2)cos(\theta_3) + sen(\theta_2)sen(\theta_3)cos(\theta_2) = cos(\theta_3)$$

Pertanto si ottiene:

$$p_{wk}^2 + p_{wz}^2 = a_2^2 + a_3^2 + 2a_2a_3cos(\theta_3)$$

da cui si ottiene:

$$cos(\theta_3) \ = \ \frac{p_{wk}^2 + p_{wz}^2 - a_2^2 - a_3^2}{2a_2a_3}$$

dove p_{wk} e p_{wz} sono le coordinate del polso O_4 del manipolatore Tinkerkit e a_2, a_3 sono le lunghezze del braccio (Arm) e dell'avambraccio (Forearm) di questo.
Di conseguenza è possibile calcolare il seno di θ_3

$$sen(\theta_3) = \pm\sqrt{1 - cos^2(\theta_3)}$$

Noi assumeremo:

$$sen(\theta_3) = -\sqrt{1 - cos^2(\theta_3)}$$

CALCOLO DELL'ANGOLO θ_2 (SPALLA)

Per calcolare l'angolo θ_2 del braccio si riprendono le relazioni:

$$\begin{cases} p_{wk} = a_2 cos(\theta_2) + a_3 cos(\theta_2 + \theta_3) \\ p_{wz} = a_2 sen(\theta_2) + a_3 sen(\theta_2 + \theta_3) \end{cases}$$

che per le formule di addizione del seno e del coseno diventano:

$$\begin{cases} p_{wk} = a_2 cos(\theta_2) + a_3 (cos(\theta_2) cos(\theta_3) - sen(\theta_2) sen(\theta_3) \\ p_{wz} = a_2 sen(\theta_2) + a_3 (sen(\theta_2) cos(\theta_3) + cos(\theta_2) sen(\theta_3)) \end{cases}$$

Riordinando i secondi termini e risolvendo il sistema nelle incognite cos(θ_2), sen(θ_2), con il metodo di Cramer si ottiene:

$$\begin{cases} p_{wk} = (a_2 + a_3 cos(\theta_3)) cos(\theta_2) - (a_3 sen(\theta_3)) sen(\theta_2) \\ p_{wz} = (a_3 sen(\theta_3)) cos(\theta_2) + (a_2 + a_3 cos(\theta_3)) sen(\theta_2) \end{cases}$$

$$cos(\theta_2) = \frac{\begin{vmatrix} p_{wk} & -a_3 sen(\theta_3) \\ p_{wz} & a_2 + a_3 cos(\theta_3) \end{vmatrix}}{\begin{vmatrix} a_2 + a_3 cos(\theta_3) & -a_3 sen(\theta_3) \\ a_3 sen(\theta_3) & a_2 + a_3 cos(\theta_3) \end{vmatrix}}$$

$$sen(\theta_2) = \frac{\begin{vmatrix} a_2 + a_3 cos(\theta_3) & p_{wk} \\ a_3 sen(\theta_3) & p_{wz} \end{vmatrix}}{\begin{vmatrix} a_2 + a_3 cos(\theta_3) & -a_3 sen(\theta_3) \\ a_3 sen(\theta_3) & a_2 + a_3 cos(\theta_3) \end{vmatrix}}$$

Risolvendo si ottiene:

$$cos(\theta_2) = \frac{p_{wk}(a_2 + a_3 cos(\theta_3) + p_{wz} a_3 sen(\theta_3)}{(a_2 + a_3 cos(\theta_3))^2 + (a_3 sen(\theta_3))^2}$$

$$sen(\theta_2) = \frac{p_{wz}(a_2 + a_3 cos(\theta_3) - p_{wk} a_3 sen(\theta_3)}{(a_2 + a_3 cos(\theta_3))^2 + (a_3 sen(\theta_3))^2}$$

Osservando la Fig.5 si ricava che:

$$(a_2 + a_3 cos(\theta_3))^2 + (a_3 sen(\theta_3))^2 = p_{wk}^2 + p_{wz}^2$$

e pertanto le soluzioni del sistema diventano:

$$cos(\theta_2) = \frac{p_{wk}(a_2 + a_3 cos(\theta_3)) + p_{wz} a_3 sen(\theta_3)}{p_{wk}^2 + p_{wz}^2}$$

$$sen(\theta_2) = \frac{p_{wz}(a_2 + a_3 cos(\theta_3)) - p_{wk} a_3 sen(\theta_3)}{p_{wk}^2 + p_{wz}^2}$$

CALCOLO DELL'ANGOLO Θ_4 (POLSO)

$$\phi = \theta_2 + \theta_3 + \theta_4$$

Per calcolare θ_4 si utilizza la relazione precedentemente introdotta:
da cui si ricava:

$$\theta_4 = \ \phi - \theta_2 - \theta_3$$

CODICE PYTHON PER IL CALCOLO DEGLI ANGOLI PER LA PRESA DELLA SPUGNETTA

AnimazioneBraccio_mac.py (3.14.7)

```python
# Python script per Blender per la presa dell'oggetto Cube e lo spostamento di questo nel cerchio sulla base

import bpy
from mathutils import Matrix
import math


from PIL import Image
import matplotlib.pyplot as plt
import numpy as np



# parametri fisici del braccio tinkerkit
d1 = 0.075 # altezza del bone Base  (75 mm)
a2 = 0.125 # lunghezza del bone Arm (125 mm)
a3 = 0.125 # lunghezza del bone Forearm (125 mm)
a4 = 0.185 # lunghezza del bone Wrist_vertical + lunghezza della pinza (185 mm)

# Dimensioni dell'oggetto Cube: Dimensions: (dx = 58 mm, dy = 28 mm, dz = 72 mm)
dx = 0.075
dy = 0.025
dz = 0.085

pez = dz/2 - d1     # altezza del centro dell'oggetto Cube - l'altezza del bone Base
phi = -3.1415/3     # angolo tra il bone Wrist_vertical e l'orizzontale, -3.1415/3  in radianti equivale a - 60°

# angoli della pinza
gripopen =  0.6981 # Angolo della pinza aperta gripopen, 0.6981 in radianti equivale a 40°
gripeng = 0.1745   # Angolo della pinza in presa gripeng, 0.1745 in radianti equivale a 14°
gripclose = 0.0    # Angolo della pinza chiusa griclose, 0.0 in radianti equivale a 0°

# angoli di rotazione dei bones nella posizione a riposo del braccio
theta1 = 0.0        # angolo del bone Base (y_R)
theta2 = 2.3561     # angolo del bone Arm (x_R), 2.3561 corrisponde a 135°
theta3 = 0.0        # angolo del bone Forearm (x_R) rispetto all'asse del bone Arm
theta4 = 0.0        # angolo del bone Wrist_vertical (x_R) rispetto all'asse del bone Forearm
theta5 = 0.0        # angolo del bone Wrist_rotation (y_R)
theta6 = gripclose  # angolo del bone Gripper (z_R) per l'apertura e la chiusura della pinza



def angoliPresaRilascioCube(x, y, z):
    # vengono determinati gli angoli  theta1, theta2, theta3, tetha4 dei bone per la presa dell'oggetto Cube

    # Determinazione di theta1, angolo di rotazione del bone Base per raggiungere l'oggetto Cube da prendere
    # math.atan2() è la funzione arcotangente con gestione quadranti
    theta1 = math.atan2(y, x)  # in radianti

    # Calcolo degli angoli theta2, theta3 e theta4 per la rotazione dei bone Arm, Forearm e Wrist_vertical, chiamando phi l'angolo  tr

    pek = pow( x**2 + y**2, 1/2)       # distanza dall'origine del centro dell'oggetto Cube

    # Calcolo della posizione (pwk, pwz) della base del bone Wrist_vertical  nel piano verticale che lo contiene, dove l'asse k è nel
    pwk = pek - a4*math.cos(phi)
    pwz = pez - a4*math.sin(phi)

    # Calcolo dell'angolo theta3 (del bone Forearm con l'asse del bone Arm)
    cos_theta3 = (pwk**2 + pwz**2 -a3**2 - a2**2)/(2*a3*a2)     #  cos_theta3 è uguale a math.cos(theta3)
    sin_theta3 = - pow( 1 - cos_theta3**2, 1/2)                     # sin_theta3 è uguale a math.sin(theta3)
    theta3 = math.atan2(sin_theta3, cos_theta3)

    # Calcolo dell'angolo theta2 che il bone Arm forma col piano orizzontale X, Y
    cos_theta2 = (pwk*(a2+a3*cos_theta3) + pwz*a3*sin_theta3)/((a2 + a3*cos_theta3)**2 + (a3*sin_theta3)**2)    #  cos_theta2 è uguale

    sin_theta2 = (pwz*(a2+a3*cos_theta3) - pwk*a3*sin_theta3)/((a2 + a3*cos_theta3)**2 + (a3*sin_theta3)**2)    # sin_theta2 è uguale

    theta2 = math.atan2(sin_theta2, cos_theta2)

    # Calcolo dell'angolo theta4 che il bone Wrist_vertical forma con l'asse del bone Forearm
    theta4 = phi - theta2 - theta3

    return (theta1, theta2, theta3, theta4)


```